

Software Engineering in Industrial Practice (SEIP)

Dr. Ralf S. Engelschall

ARCHITECTURE FUNDAMENTALS

Architecture Maxims

TECHNISCHE UNIVERSITÄT MÜNCHEN

AF 04.1

Business Drives (BD)
Trigger and support the business with technological feasibilities, but always understand the business domain and its demands and align your architecture accordingly.

Component Orientation (CO)
Master complexity in your architecture through stringent bottom-up use of components on all scopes and zoom-levels, loose coupling between and strong cohesion within components.

Separation of Business and Technology (BT)
Strictly separate the business, i.e., domain-specific, aspects from the technological, i.e., infrastructural, aspects. Furthermore, ensure the explicit visibility of domain concepts.

Adequate Description (AD)
Provide as much stakeholder-directed architecture description as necessary, and as little as possible.

Use-Case Driven Design (UC)
Design is how it works and runs, so support your customers in their daily work by directly designing your architecture along their domain-specific use-cases.

Analytical and Creative Act (AC)
Recognize that every good architecture is based on both analytical engineering (structure) and creative artistic (beauty) aspects.

Balance Principles Against Requirements (PR)
By weighing them against one another, find a reasonable balance between fundamental architecture principles and your particular non-functional requirements.

Insights through Views & Aspects (VA)
Give insights into your architecture through carefully selected stakeholder-directed separate views and aspects. Express each with the most suitable graphical or textual language.

Proven Basis (PB)
Never start an architecture from scratch. Instead start from proven reference architectures, patterns and templates. Even if, after some iterations, no initial content is left.

Don't Be Too Clever (TC)
Don't be too clever or tricky, both in your higher-level architecture and lower-level design aspects.

Design for Failure Case (DF)
Murphy was an architect: everything which can fail will sometime ultimatively fail. Hence, already design for the failure case (think: "pessimistic").

Continuous Compliance (CC)
Continuously check through qualitative inspections and quantitative measurements whether your architecture and the non-functional requirements are followed and do not drift apart.

No Silver Bullet (SB)
There is no "one-size-fits-all" architecture, so accept that although you should reuse proven architecture aspects as much as possible, you will always need to individualize your designs.

Simplicity Trumps (ST)
Create solution parts as simple as possible and only as complex as necessary. And remember: simplicity before generality, use before reuse!

Design to Change (DC)
Time changes everything, so your solution is already legacy at the first day of release. Hence, already design for its change (think: "agile").

Integration-Figure Architect (IF)
Recognize that you, the architect, are the central integrating figure, having to bridge between the business and technology spheres of people.

Stepwise Refinement (SR)
Start with the "big picture" and perform a stepwise top-down refinement of your architecture by going from coarse to fine aspects.

Perfect is the Enemy of Good Enough (GE)
Beware of the perfection pitfall and design your architecture only as good as necessary and not as good as ultimately possible.

Explicit Decisions (ED)
Record your major architecture decisions and rationales by taking into account and back-referencing the non-functional requirements.

Eat Your Own Dog-Food (OF)
Theory and practice usually differ. Hence it is vital that every architect has good hands-on experience and must both be able to craft the solution and is willing to hypothetically intensively use it himself.

Intellectual Content: Version 1.0 (4.0.2020-04-14-09), Authored 2009-2020 by Dr.-Ing. Ralf S. Engelschall
Graphics/Illustration: Version 1.0 (8.0.2020-04-14-08), Copyright © 2012-2020 Dr.-Ing. Ralf S. Engelschall, http://engelschall.com, All Rights Reserved.
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture comments only.

In der IT Architecture folgt man **Architecture Maxims**, welche grundlegende Richtlinien sind. Man kennt 20 Maxime. Der Architekt sollte den Maximen immer folgen und sie nie brechen.

Beachte: **Proven Basis** und **No Silver Bullet** sagen, daß bei einer Architektur immer auf einer bewährten Basis (z.B. einer Referenz-Architektur) aufgesetzt werden muss, es aber gleichzeitig klar sein muss, daß man diese nicht 1:1 verwenden kann, sondern immer erst anpassen muss.

Beachte: **Stepwise Refinement** und **Component Orientation** sagen, daß vom zeitlichen Prozess her (und aus Gründen der Risikominimierung) man immer vom Groben zum Feinen geht, während die Ergebnisse eine stringente Komponenten-Orientierung aufweisen, wo kleine Komponenten hierarchisch in größeren Komponenten integriert sind.

Beachte: Während man als IT-Architekt alle anderen Maxime eigentlich nur akzeptieren muss, ist **Simplicity Trumps** von einer anderen Qualität: Nichts in der IT ist wirklich einfach. Wenn etwas einfach wirkt, versteht man meist nur noch nicht genügend davon. Oder jemand hat richtig viel investiert, um es einfach wirken zu lassen. **Simplicity Trumps** bedeutet genau dies zu tun: inhärent komplexe Dinge erst wieder einfach werden zu lassen.

Fragen

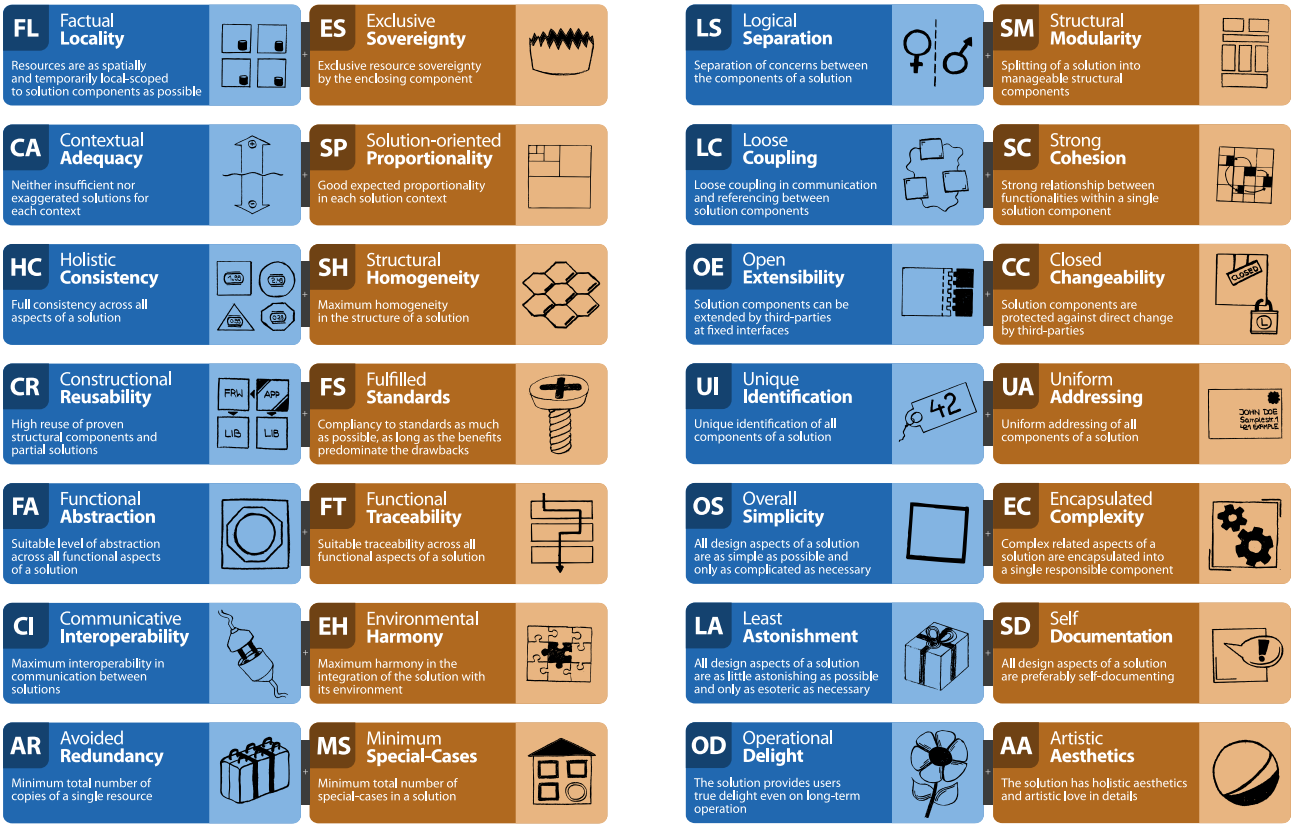
- Aus Gründen der Risikominimierung sollte der IT-Architekt beim **Stepwise Refinement** schrittweise immer wie vorgehen?

ARCHITECTURE FUNDAMENTALS

Architecture Principles

TECHNISCHE UNIVERSITÄT MÜNCHEN

AF 04-2



Intellectual Content: Version 1.0.15 (2012-12-10), Authored 2010-2012 by Dr. Ralf S. Engelschall
Graphic/Illustration: Version 1.0.13 (2011-04-06), Copyright © 2011-2012 by Ralf S. Engelschall <http://engelschall.com>, All Rights Reserved.
Unauthorized Reproduction Prohibited, Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture comments only.

In der IT Architecture folgt man **Architecture Principles**, welche grundlegende Prinzipien und Vorgehen zusammenfassen. Man kennt 28 Prinzipien, die in 14 Pärchen gruppiert werden können, da immer zwei Prinzipien inhaltlich eine starke Nähe aufweisen. Der Architekt soll den Prinzipien generell folgen, darf sie aber verletzen, solange er einen guten Grund dafür hat. Der beste Grund wäre eine spezielle projektspezifische Anforderung.

Beachte: Das Prinzip **Logical Separation** (aka **Separation of Concern**) ist eines der wichtigsten, da aus ihm etliche andere Prinzipien fast automatisch folgen, unter anderem z.B. **Structural Modularity**.

Hinweis: Die Prinzipien **Loose Coupling** und **Strong Cohesion** sind in der Literatur als das Kombi-Prinzip "Loose Coupling, Strong Cohesion" bekannt. Die Prinzipien **Open Extensibility** und **Closed Changeability** sind in der Literatur als das Kombi-Prinzip "Open-Close" bekannt.

Beachte: Das Prinzip **Overall Simplicity** ist eines der schwersten umzusetzenden Prinzipien, da nichts in der IT wirklich einfach ist. Alles erscheint nur so lange einfach, solange man noch nicht genügend davon versteht. Danach muss man es erst wieder mühsam neu "einfach" machen. Das ist die Kunst bei Architektur: schwierige Dinge zu vereinfachen! Wenn etwas nicht viel weiter vereinfacht werden kann und immer noch gewisse Komplexität aufweist, kann man es zumindest mit dem Prinzip **Encapsulated Complexity** versuchen zu verschatten.

Fragen

- Zählen Sie mindestens 4 wesentliche **Architecture Principles** auf!

ARCHITECTURE FUNDAMENTALS

Component Design

TECHNISCHE UNIVERSITÄT MÜNCHEN

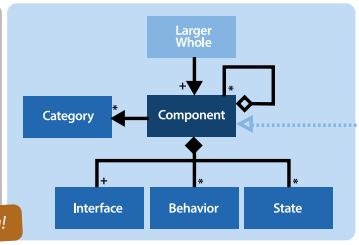
AF 05.1

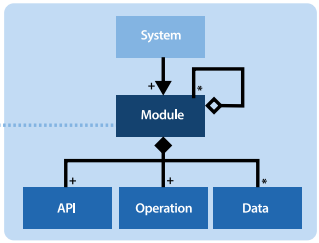
Definition of a Component (of a Larger Whole):
a **know-how encapsulating**, potentially **reusable** and **substitutable unit of hierarchical composition** with **explicit context dependencies**, which **hides the complexity** of its optional **behavior** and **state** realization behind **small contractually specified interfaces**, defines its **added value** in terms of **provided** and **consumed** interfaces and optionally belongs to zero or more **categories** of similar units.

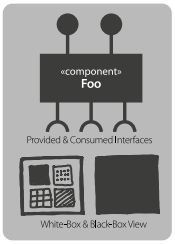
Definition of a Module (of a System):
a know-how encapsulating, potentially reusable and substitutable **source-code** unit of hierarchical composition with explicit context dependencies which hides the complexity of its **operation** and **data implementation** behind small contractually specified **Application Programming Interfaces (API)**, defines its added value in terms of provided and consumed **APIs** and optionally belongs to zero or more categories of similar units.

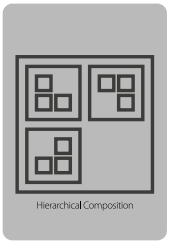
Example Categories of Components:
- Namespace
- Directory, File
- Configuration, Section, Directive
- Host, Virtual Machine, Container
- Process Group, Process, Thread
- Application, Microservice, Program
- Package, Class, Function
- Database, Schema, Table, Record
- Datamodel, Entity Group, Entity
- User Interface, Dialog, Widget

Any group of anything!









How to find Components (or Modules)?

DCA Domain Concept Abstraction
Model domain concepts as entity components and then group at higher levels.

SOC Separation of Concerns
Build components for clearly distinct concerns.

USE Reusability Potential
Decide on components based on their reusability potential.

UCC Use-Case Clustering
Build domain components for each use-case or each logical use-case cluster.

SRP Single Responsibility Principle
Build components for clearly distinct responsibilities.

DCC Divide & Conquer Complexity
Master overall complexity by splitting larger things into smaller things.

DDD Domain-Driven Design
Model domain "Bounded Contexts" through DDD and derive components from them.

CNC Coupling and Cohesion
Decide on components based on their loose coupling and strong cohesion.

CCC Cross-Cutting Concerns
Build common cross-cutting concerns as cross-cutting components.

OOD Object-Oriented Design
Model Object-Oriented Design entities (and/or OOP constructs) as components.

DEP Dependency Encapsulation
Decide on components based on their encapsulation of dependencies.

PAT Architecture Patterns
Build inner components to comply to outer structure, slicing and clustering architectures.

Intellectual Content: Version 1.0.3 (2020-04-28), Authored 2017-2020 by Dr. Ralf S. Engelschall
Graphic/Layout/Illustration: Version 1.0.2 (2020-10-23), Copyright © 2017-2023 Dr. Ralf S. Engelschall <http://engelschall.com>, All Rights Reserved.
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture comments only.

In der Software Architecture dreht sich alles um Komponenten (**Components**) und Schnittstellen (**Interfaces**). Deshalb ist **Component Design** eine zentrale Aufgabe des Architekten.

Eine Component **kapselt** ein bestimmtes **Know-How**, ist **potenziell wiederverwendbar** und **ersetzbar**. Eine Component besitzt ein **Verhalten** und einen **Zustand** und verbirgt die interne Komplexität von beidem hinter "kleinen" **vertraglichen Schnittstellen**. Sie stellt ihren Mehrwert über die Differenz bereitgestellter und konsumierter Schnittstellen bereit. Sie kann als **Whitebox** oder **Blackbox** betrachtet werden, abhängig davon, ob man ihre internen Details von außen sieht oder nicht. Components sind hierarchisch angeordnet, gehören eventuell zu bestimmten **Kategorien** und besitzen **explizite Abhängigkeiten** untereinander.

Man unterscheidet zwischen dem allgemeineren Konzept der **Component** ("any group of anything") und dem spezielleren Konzept des (über Source Code definierten) **Module**.

Components findet man über zahlreiche Wege. Die meisten leiten sich direkt aus bestehenden Methoden, Prinzipien oder Mustern ab. Die beiden wichtigsten Wege für einen Komponentenschnitt in der Praxis sind: **Separation of Concerns** (welches einmalige Anliegen bzw. welche einmalige Aufgabe hat die Komponente?) und **Single Responsibility Principle** (welche einmalige Verantwortung hat die Komponente?).

Fragen

- Zählen Sie mindestens 7 Eigenschaften/Aspekte auf, die eine Komponente besitzt!
- Was sind die beiden wichtigsten Wege, um in der Praxis einen Komponentenschnitt zu finden?

ARCHITECTURE FUNDAMENTALS

Interface Design

TECHNISCHE UNIVERSITÄT MÜNCHEN

AF 05.2

Definition of an Interface: well-defined shielding and abstracting **boundary** of a passive, providing **component**, consisting of one or more distinguished, **outside-in** designed, **interaction endpoints**, each accessed and controlled by active, consuming components through the **exchange** of **input/output information** and operating under a certain **syntactical** and **semantical contract**.



Endpoint: Name, Directive, Command, Function, Method, Procedure, Address, Port, URL, Dialog, ...
Exchange: Option, Argument, Parameter, Return Value, Result, Request/Response Message, Error/Exception, Interaction, ...
Contract: Syntax, Pre-Condition, Invariant, Post-Condition, Side-Effect, Idempotence, Determinism, Functionality, ...

Types of Software Interfaces

API Application Programming Interface
Example: `foo("bar", 42)` (call and use)



SPI Service Provider Interface
Example: `register("foo", (x,k) => ...)` (extend and implement)



SCI Startup Configuration Interface
Examples: INI, Java Properties, TOML, YAML, JSON, XML, etc.



BPI Batch Processing Interface
Examples: Unix at(1), Unix ts(1), GNU Batch, Spring Batch, Java Batch, SAP LO-BM, etc.



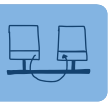
CLI Command-Line Interface
Example: `foo -x --bar=baz quux`



GUI Graphical User Interface
Examples: Windows/WPF, macOS/Cocoa, KDE/Qt, GNOME/GTK



RNI Remote Network Interface
Examples: GraphQL-IO, HTTP/REST, SOAP, RMI, WebSockets, AMQP, MQTT, etc.



Characteristics of Good Interfaces

AP Appropriate & Proportional
Appropriate to consumer requirements, proportionate to provider functionality.



SA Shielding & Abstracting
Shields from direct access, abstracts and hides implementation details.



IE Inviting & Expressive
Invites through "outside – in" design, powerful in expressiveness.



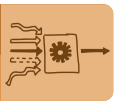
IF Intuitive & Foolproof
Intuitive to grasp and use, hard to misuse.



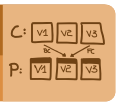
OC Orthogonal & Concise
Supports combinatorial use-cases, causes minimum boilerplate.



TP Tolerant & Predictable
Tolerant on input, predictable on output.



EC Extensible & Compatible
Easy to extend for providers, backward/forward-compatible for consumers.



Selected Interface Design Patterns

IVF Interface Version & Features
Provide version and feature information for algebraic comparison and feature detection.



2LF Leaky Two-Layer Facade
Provide higher-level convenient use-case and lower-level orthogonal feature interface.



EVE Event Emitter
Emit events to previously registered, interested consumers.



CTX Multi-Context
Use contexts to distinguish between different usage scenarios and to carry common info.



CEF Configure-Execute Flow
Spread use-cases onto a flow of configuration exchanges and a final executional exchange.



IOC Inversion Of Control
Invert control on asynchronous operations via callbacks, promises or async. mechanisms.



HMR Human/Machine Responses
Support humans and machines in outputs through both description and parsing-free info.



Intellectual Content: Version 1.0.0 (2022-12-20), Authored 2019-2022 by Dr. Ralf S. Engelschall
Graphics/Illustration: Version 1.0.8 (2022-12-20), Copyright © 2021-2022 Dr. Ralf S. Engelschall <http://engelschall.com>, All rights Reserved.
Unauthorized Reproduction Prohibited. Licensed for Technische Universität München (TUM) for reproduction in Computer Science lecture comments only.

Eine Schnittstelle (**interface**) ist eine wohldefinierte (**well-defined**), abschirmende (**shielding**), abstrahierende (**abstracting**) Berandung (**boundary**) einer passiven bereitstellenden Komponente (**component**), welche aus einem oder mehreren klar unterscheidbaren Interaktionsendpunkten (**interaction endpoints**) besteht.

An jedem Interaktionsendpunkt wird von einer passiven bereitstellenden Komponente auf eine aktive, konsumierende Komponente durch den Austausch (**exchange**) von **input/output** Informationen (**information**) zugegriffen und dieser wird unter einem bestimmten syntaktischen (**syntactical**) und semantischen (**semantical**) Vertrag (**contract**) betrieben.

Es gibt zahlreiche Arten von Schnittstellen, die alle dieser Definition entsprechen. "Gute" Schnittstellen haben darüber hinaus bestimmte Eigenschaften/Charakteristiken. Die vier der besten Eigenschaften sind: **Proportional** (die Schnittstelle ist kleiner und in der Größe proportional zur dahinterliegenden Funktionalität), **Expressive** (die Schnittstelle stellt ein leistungsfähiges Programmiermodell zur Verfügung), **Orthogonal** (die Schnittstelle erlaubt kombinatorische Use-Cases) und **Concise** (die Schnittstelle erzeugt wenig "Boilerplate Code" bei der Nutzung).

Es gibt zahlreiche Software Pattern für Schnittstellen. Ein interessantes Muster ist die **Leaky Two-Layer Facade**, bei dem eine Bibliothek zwei Schnittstellen besitzt: eine obere, bequeme und Use-Case-bezogene Schnittstelle und eine untere, orthogonale Feature-bezogene Schnittstelle. Der Witz ist, daß die obere durch alleine die untere Schnittstelle implementiert wird und daß die untere Schnittstelle "durchscheint" ("leaky" bzw. Open Layering).

Fragen

- Zählen Sie mindestens 8 Eigenschaften/Aspekte auf, die eine Schnittstelle definieren!
- Zählen Sie mindestens 4 Eigenschaften/Charakteristiken von guten Schnittstellen auf!